COURSE PROJECT

TRUCK AND TRAILER ROUTING WITH SATELLITE DEPOTS

Combinatorial optimization
Greedy construction, local search, and metaheuristics

Andon Tchechmedjiev
Three four-hour working sessions

IMT Mines Alès
École Mines-Télécom

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

COURSE PROJECT

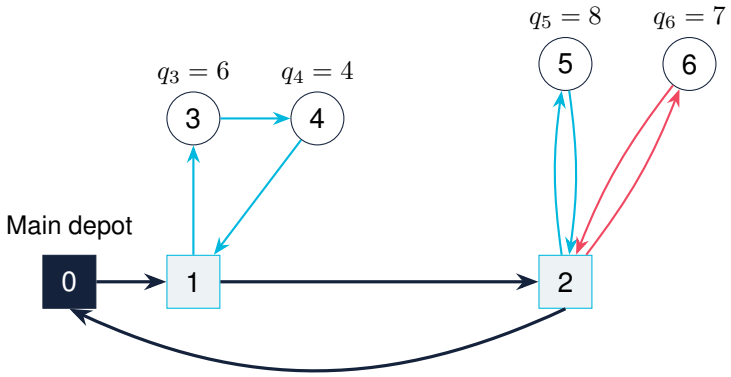
METAHEURISTICS FOR TRUCK AND TRAILER ROUTING

1 THE PROJECT AND THE ROUTING PROBLEM

In teams of four or five, design, implement, and evaluate a metaheuristic for the Single Truck and Trailer Routing Problem with Satellite Depots (STTRPSD). Build your method over three four-hour working sessions: first a greedy constructor, then local search, and finally a more complex metaheuristic that builds on both. Keep all three versions so that you can measure what each stage contributes.

You will receive ten benchmark instances for development. Your programme will also be assessed on ten other instances with similar characteristics. Every timed run has a fixed budget of **30 seconds**. Your explanation of the algorithm and your analysis of the experiments matter alongside the quality of the routes. You can reuse ideas from the nature-inspired metaheuristic paper that your group previously presented.

The setting comes from Villegas et al. [1]. A truck with a detachable trailer starts at a main depot. Customers are accessible only to the truck alone. The vehicle can leave its trailer at one of several satellite depots, make one or more trips to customers, return to the parked trailer after each trip, and then move on. Applications include milk collection where narrow access roads prevent the trailer from reaching farms. We use a delivery interpretation below; the route structure and capacity conditions are the same for pure collection.



Dark lines: truck and trailer. Coloured lines: truck only.

The decisions are coupled. Selecting a nearby satellite may shorten truck trips but increase the main tour. Packing more customers into a truck trip reduces returns to the satellite but changes customer order and may violate capacity. Your algorithm must make these choices together.

The required solution has one main tour through selected satellites and a set of truck-only trips. Each customer appears exactly once. Every truck trip returns to the same satellite at which it began. The truck and trailer return to the main depot at the end. The objective is the sum of all travelled distances.

2 THE ROUTING MODEL

2.1 Input and assumptions

Let 0 be the main depot, $\mathcal{S}$ the candidate satellites, and $\mathcal{C}$ the customers. These sets are disjoint, with $\mathcal{C} \neq \varnothing$ and $\mathcal{S} \neq \varnothing$. Let $V = \{0\} \cup \mathcal{S} \cup \mathcal{C}$. Customer $i \in \mathcal{C}$ has demand $q_i > 0$. The truck and trailer have capacities $Q_V > 0$ and $Q_T \geq 0$.

For the supplied benchmark, each node has coordinates (X_i, Y_i) and

$$d_{ij} = \sqrt{(X_i - X_j)^2 + (Y_i - Y_j)^2}, \qquad i, j \in V.$$

Use these distances **without rounding to integers**. Displayed results may be rounded, but objective calculations and rankings use the underlying values. The original benchmark README is included in `sources/benchmark_README.txt`.

The instances satisfy

$$\max_{i \in \mathcal{C}} q_i \leq Q_V, \qquad \sum_{i \in \mathcal{C}} q_i \leq Q_V + Q_T.$$

The first condition makes each customer serviceable by the truck. The second makes the overall demand compatible with the single vehicle. Satellites are parking and transfer locations, with no individual stock or service-capacity limit. There are no time windows, route-duration limits, satellite-opening costs, or handling costs in this project.

2.2 A feasible solution

A main tour is an ordered sequence

$$M = (0, s_1, \ldots, s_k, 0), \quad k \geq 1, \quad s_j \in \mathcal{S},$$

in which the s_j are distinct. A truck trip r has a root satellite $s(r)$ and an ordered, nonempty customer sequence $(i_{r,1}, \ldots, i_{r,m_r})$. Its complete traversal is

$$(s(r), i_{r,1}, \ldots, i_{r,m_r}, s(r)).$$

The following conditions define feasibility.

1. Every root $s(r)$ occurs in the main tour.
2. Every customer occurs in exactly one position of exactly one truck trip.
3. Each trip satisfies $\sum_{h=1}^{m_r} q_{i_{r,h}} \leq Q_V$.
4. The main tour visits only the main depot and satellites. Truck-trip interiors contain only customers.

Several trips may use the same satellite. The one truck performs them sequentially while the trailer remains parked, then collects the trailer before continuing along M. A truck trip cannot finish at another satellite. Direct customer trips rooted at depot 0 are outside this benchmark variant.

Capacity is checked per truck trip. The total demand assigned to a satellite may exceed Q_V because the truck can return and reload there. The overall bound is $Q_V + Q_T$, not Q_T alone. The model allows transfers at a satellite; no additional satellite capacity is imposed. An unnecessary satellite visit is allowed and charged, although it can be removed without increasing Euclidean distance.

2.3 Objective

Writing $s_0 = s_{k+1} = 0$, minimize

$$f(M, \mathcal{R}) = \underbrace{\sum_{j=0}^{k} d_{s_j s_{j+1}}}_{\text{main tour}} + \underbrace{\sum_{r \in \mathcal{R}} \left(d_{s(r) i_{r,1}} + \sum_{h=1}^{m_r - 1} d_{i_{r,h} i_{r,h+1}} + d_{i_{r,m_r} s(r)} \right)}_{\text{all truck trips, including their return legs}}.$$

An infeasible solution has no admissible objective value. The scorer never accepts a claimed objective in place of checking the routes.

2.4 What a search operator changes

Your programme can work directly with the main tour and the lists of truck trips defined above. A move may reorder customers within a trip, move or exchange customers between trips, change a trip's root, merge or split trips, or alter the main tour. Evaluate every distance term affected by the move and check that the resulting routes remain feasible.

For example, moving a trip to another satellite changes its first and last truck edges. It may also add a satellite to the main tour or remove one that no longer serves customers. A short truck trip can therefore be part of an expensive overall solution. The objective above gives a common basis for the greedy constructor, local search, and final metaheuristic.

3 INSTANCES AND THE SOLUTION INTERFACE

3.1 Benchmark files

The supplied data originate from the 32-instance benchmark of Villegas et al. The original files use the following layout, supported by the converter:

```
n p_plus_one
truck_capacity trailer_capacity
main_depot_x main_depot_y
... p rows of satellite coordinates ...
... n rows of customer_x customer_y demand ...
```

The count on the first line includes the main depot. Canonical identifiers are 0 for that depot, $1, \ldots, p$ for satellites, and $p+1, \ldots, p+n$ for customers. A few coordinates exceed 100; preserve them as supplied. Instances in the course package are already converted to JSON. The original data and their conversion records are retained by the instructor.

Each JSON instance contains `schema_version`, `name`, `truck_capacity`, `trailer_capacity`, a `depot` object, lists of `satellites` and `customers`, and `distance: {"type":"euclidean"}`. Nodes have integer `id` and numerical `x,y`; customers additionally have `demand`. Optional `metadata` records provenance. Exact schemas and the complete toy instance are included.

3.2 A complete solution example

The file `examples/toy_solution.json` contains:

```
{
  "schema_version": 1,
  "instance": "toy",
  "main_route": [0, 1, 2, 0],
  "truck_routes": [
    {"satellite": 1, "customers": [3, 4]},
    {"satellite": 2, "customers": [5]},
    {"satellite": 2, "customers": [6]}
  ]
}
```

The satellite at the beginning and end of each truck trip is implicit. Do not insert it in `customers`. The `instance` value must exactly match the instance's `name`. Integers identify nodes; array positions do not. The `main_route` explicitly repeats the depot at the two ends. An optional `metadata` object is accepted. Unknown fields, including a top-level claimed objective, are rejected.

3.3 Checking the example by hand

Use depot $0 = (0, 0)$, satellites $1 = (2, 0)$ and $2 = (8, 0)$, and customers $3 = (2, 3)$, $4 = (4, 3)$, $5 = (8, 4)$, $6 = (10, 4)$. Their demands are $6, 4, 8, 7$, with $Q_V = 10$ and $Q_T = 15$. The total demand is 25, within the combined capacity. The three truck-trip loads are $10, 8, 7$.

$$\begin{aligned} f_{\text{main}} &= 2 + 6 + 8 = 16, \\ \ell_{(1,3,4,1)} &= 3 + 2 + \sqrt{13}, \quad \ell_{(2,5,2)} = 8, \quad \ell_{(2,6,2)} = 2\sqrt{20}, \\ f &= \boxed{41.54982318546315\ldots}. \end{aligned}$$

Putting customers 5 and 6 in one trip would violate capacity because $8 + 7 = 15 > 10$. Serving a customer twice, omitting one, or using a satellite absent from the main tour also makes the solution invalid.

4 EXECUTION AND COMPARISON PROTOCOL

4.1 The programme contract

Your programme must read a supplied JSON instance, accept a random seed and a time budget, and write one JSON solution file. You may use any implementation language. Configure an argument-list command for the harness with placeholders `{instance}`, `{solution}`, `{seed}`, and `{time_limit}`. The starter illustrates this interface:

```
python -m sttrpsd baseline --instance examples/toy.json \
  --solution my_solution.json --seed 0 --time-limit 30
python -m sttrpsd score --instance examples/toy.json \
  --solution my_solution.json
python -m sttrpsd benchmark --config configs/demo.json --output demo_run
```

The package uses Python 3.10 or later and the standard library. Run these commands from the package root. Your own dependencies must be documented separately. Write the output before the budget expires, preferably by replacing a temporary file atomically. Use time left for output and clean termination. The harness does not accept a partial file from a timed-out process.

4.2 Assessment settings

The time budget is fixed at **30 seconds of wall-clock time per run**, including process startup. Installation and compilation take place beforehand. The prepared assessment configuration uses ten instances and three common seeds $(0, 1, 2)$. Runs execute sequentially on the same machine, pinned to one CPU where supported. The instructor will confirm the common seed set and computing environment before assessment; the 30-second limit applies to every team and every method in the comparison.

Compare your greedy constructor, local search, and final metaheuristic under this same 30-second limit. A method may finish earlier. Record its actual runtime and use the returned solution; an early finish does not justify a longer budget for another method. The final assessment evaluates the team's submitted metaheuristic, while your report explains how it compares with the first two stages.

A run is valid only if the process finishes successfully within the budget and its output passes all feasibility checks. A timeout, crash, missing output, malformed JSON, or infeasible solution earns zero performance credit for that run. The scorer permits numerical capacity slack of $10^{-9} \max(1, Q)$ for floating-point calculations; it is not permission to exceed the capacity deliberately. The benchmark's integer demands make capacity comparisons exact in normal use.

4.3 Performance mark out of eight

For assessment instance i and seed r, let f_{gir} be group g's valid objective and b_{ir} the best valid objective among the ranked groups in that same instance–seed cell. Demonstration or reference solvers marked as unranked are excluded from b_{ir}. Define

$$u_{gir} = \begin{cases} \min(1, b_{ir}/f_{gir}) & \text{if the run is valid and } f_{gir} > 0, \\ 1 & \text{if the run is valid and } b_{ir} = f_{gir} = 0, \\ 0 & \text{if the run fails or no ranked run is valid.} \end{cases} \qquad K_g = \frac{8}{|I||S|} \sum_{i \in I} \sum_{r \in S} u_{gir}.$$

Here I is the set of ten assessment instances and $S = \{0, 1, 2\}$. Every scheduled run remains in the denominator, including failures. Thus instances and seeds have equal weight, and a single long-distance instance cannot dominate the mark. A cost of 110 against a best observed cost of 100 gives $u = 100/110 \simeq 0.9091$. If that ratio holds for all runs, the mark is approximately $7.27/8$. This is a relative performance mark, not an optimality certificate.

The leaderboard sorts by K_g and gives tied scores a shared rank within the documented tolerance. It also reports feasibility, objectives, and execution times. Best-observed references depend on the compared cohort. For development, the same report helps you compare configurations of your own method; official marks use only the instructor's assessment run.

5 WORK PLAN AND ORAL EXCHANGES

5.1 Session objectives

Session 1: implement a greedy constructor.
Understand the route structure, check the toy example, and run the scorer. Choose a route representation and a greedy rule for selecting satellites or adding customers to trips. Implement that rule, explain how it maintains feasibility, and obtain valid solutions on the practice instances. Save the constructor as a runnable version and record its objectives and runtimes.
Session 2: implement local search.
Starting from your greedy solution, implement one or two neighbourhoods and an improvement rule. Suitable first moves are intra-trip 2-opt, customer relocation, and customer exchange. Explain which distance terms change and how you check capacity and coverage. Run descent until no accepted improving move remains or the budget expires. Compare greedy construction alone with construction followed by local search, keeping the same instances, seeds, and 30-second limit.
Session 3: implement a more complex metaheuristic.
Build on the constructor and local search with an evolutionary, population, swarm, or another justified metaheuristic. Explain the mechanism that explores beyond one local optimum and how it works with your route representation. Implement a manageable version, then compare it with both earlier stages under the same 30-second limit. Use the results to explain improvements, failures, and stochastic variation, and verify that the final programme runs through the required interface.

The paper studied for your earlier pitch provides a starting point for the third session. A genetic algorithm might initialize a population from greedy constructions, vary customer sequences, and improve selected candidates with local search. A swarm or ant-colony method must likewise explain how its internal state produces feasible discrete routes and how local improvement is used. Choose a small coherent implementation that fits the session; reproducing every feature of a published algorithm is not expected.

By the end of the project, every team member should be able to trace the greedy choices, explain an improving move and a local optimum, describe the metaheuristic's exploration mechanism, and interpret the measured difference between the three stages. Use the optional formulation in Appendix A if it helps you relate these routing decisions to linear programming. Implementing an exact solver is not required.

5.2 A common timetable

Time	Activity
08:30–08:45	Briefing or progress review
08:45–10:30	Team development and experiments
10:30–10:45	Break
10:45–11:30	Team development and experiments
11:30–12:11	Group exchanges; other groups continue working
12:11–12:40	Apply feedback and complete the session's work
12:40–12:45	Save the checkpoint and experiment log

The 08:30–12:45 interval contains four working hours and a fifteen-minute break. Each exchange lasts five minutes, with one minute to switch groups. Seven exchanges therefore take 41 minutes, including six transitions.

5.3 Oral order and participation

Exchange	Session 1	Session 2	Session 3
11:30–11:35	Group 4	Group 5	Group 6
11:36–11:41	Group 1	Group 3	Group 7
11:42–11:47	Group 2	Group 7	Group 5
11:48–11:53	Group 7	Group 6	Group 2
11:54–11:59	Group 6	Group 4	Group 3
12:00–12:05	Group 3	Group 1	Group 4
12:06–12:11	Group 5	Group 2	Group 1

The paper presentations already took place in group-number order. Across all four exercises, every group has average position 4 and four distinct positions. The first groups are 1, 4, 5, and 6.

No computers, phones, or notes during the exchange. Every member answers questions about the current method, a personal contribution, and a result or difficulty. The restriction applies to the oral exchange; normal project work uses computers. Other groups keep working while one group is assessed.

6 ASSESSMENT AND THE REPORT

6.1 Final mark

Let $P \in [0, 20]$ be participation, $C \in [0, 4]$ code, $R \in [0, 8]$ report, and $K \in [0, 8]$ benchmark performance. The final project mark is

$$\boxed{G = \frac{P + C + R + K}{2} \quad \text{out of 20.}}$$

Participation combines the existing paper-presentation mark out of five and three session marks out of five. A notation such as "code /4" identifies its scale; it does not mean dividing the code mark by four inside this formula. For example, $P = 16$, $C = 3$, $R = 6$, and $K = 7$ gives $G = 16/20$.

Participation reflects each student's oral understanding and contribution. Code, report, and benchmark marks apply to the team's shared submission. The existing paper-presentation mark is retained.

Session exchange rubric	/5
Understanding of the problem, feasibility, and current method	2
Explanation of personal work and decisions during the session	2
Critical interpretation of a test, result, limitation, or next step	1

Code rubric	/4
Reliable input, output, and conformity to the solver interface	1
Correct feasibility handling and meaningful verification	1
Clear implementation of the documented method	1
Reproducible execution, dependencies, seeds, and stopping behaviour	1

6.2 Required report structure

Use the following five sections, in this order. There is no additional mandatory page count; the fifth section occupies one page.

1. **Method description (2 marks).** Explain the encoding, greedy construction, local-search neighbourhoods and improvement rule, then the final metaheuristic's exploration, selection or acceptance mechanism. State how the three stages fit together, their parameters, and their stopping rules. Pseudocode should make the overall algorithm understandable. Cite borrowed ideas and explain your adaptations.
2. **Benchmarking and result presentation (2 marks).** Describe hardware, software, the fixed 30-second limit, seeds, and configurations. Report per-instance feasibility, mean and best objective, dispersion, and actual runtime for the greedy constructor, local search, and final metaheuristic. Use the same protocol for all three. Label every table and figure so it can be read independently.
3. **Analysis and conclusions (2 marks).** Explain what local search adds to greedy construction and what the metaheuristic adds to local search. Use the results to identify where each improvement succeeds or fails. Discuss stochastic variation and limitations. Distinguish an observed improvement from a claim of general superiority.
4. **Instructions to run the programme (1 mark).** Give dependencies and versions, installation/build commands, an exact example invocation, and the files it produces. Explain how to reproduce the reported experiments from a clean directory.
5. **Use of generative AI (1 mark, one page).** State the tools and models used, their purposes, the important exchanges and iterations, and how outputs were checked. Identify each student's actual contributions to conceptualization, overall direction, code, experiments, analysis, and writing. Give concrete examples of suggestions you accepted, changed, or rejected. If no generative AI was used, state this and describe the division of work.

7 IMPLEMENTATION ADVICE AND SUBMISSION

7.1 A manageable development path

The supplied constructor provides an executable reference for the interface and scorer. It chooses a satellite, groups customers into capacity-feasible truck trips, and constructs their visit order. In the first session, implement your own greedy rule and explain why each choice is feasible. A useful first version can select one satellite, repeatedly add the nearest feasible unserved customer, and start another trip whenever the current truck cannot accept a remaining customer. More elaborate satellite choices can follow once this version works.

Begin with a representation that is easy to validate. A direct representation can store the ordered satellites and a list of truck trips at each satellite. Another option is a permutation of customers with a decoder that

inserts trip boundaries and chooses satellites. In both cases, state which decisions your search explores and which are fixed by the decoder.

In the second session, add a small local-search procedure around this constructor. Generate a neighbouring solution, check feasibility, compute its complete cost, and accept an improvement. First-improvement descent accepts the first improving move found; best-improvement descent compares all moves in the chosen neighbourhood before selecting one. Either is suitable if you explain the choice. Start with one reliable neighbourhood, then add another if time permits. Use incremental cost evaluation only after comparing it with complete rescoring on small examples.

In the third session, reuse those components inside a more complex search. For a genetic or other population method, specify initialization, the encoding of an individual, variation, selection, and the role of local search. Explain how variation avoids duplicate or missing customers, or how a decoder or repair procedure restores feasibility. For a swarm method, explain how its updates become discrete route changes rather than assuming that numerical position updates are already valid tours. Use the paper pitch to guide the mechanism and keep the implementation small enough to test within the session.

Reserve part of the final session for the three-way comparison and fresh-launch checks. Keep the greedy and local-search versions available through separate commands or documented options. Record enough information during each session to describe the method and its development in the five required report sections.

7.2 Checks that reveal real errors

- Check a hand-computable solution, including every return edge.
- Try a trip exactly at capacity and one just above it.
- Check missing and duplicated customers, unknown IDs, and an unvisited root satellite.
- Check several trips at one satellite and a single-customer trip.
- Compare objective changes predicted by an operator with full rescoring.
- Run the final command from a fresh directory with its documented dependencies and output path.
- Repeat stochastic experiments with recorded seeds. Separate tuning decisions from the final comparison.

The self-contained HTML report shows the leaderboard, run statuses, distances, times, and route drawings. Use the per-run view to investigate failures. An error message is evidence that a candidate was rejected, not a soft penalty that can be traded against distance.

7.3 What to submit

Submit your source code, an environment/dependency description, the final metaheuristic command or harness configuration, your report, and the experiment results needed to reproduce its main tables. Include runnable greedy and local-search versions or options, the final parameter settings, and the seeds. Put each team's work in a clearly named directory. The instructor will announce the hand-in date and platform. All execution needed for assessment must work without an interactive prompt or a network call.

The ten supplied instances support development. The ten assessment instances are selected separately from the original benchmark and supplied to your programme at assessment. Write a general solver: solution lookup tables and instance-specific hard-coded routes do not demonstrate the required optimization method. The programme must handle the supplied instance as data.

REFERENCES

[1] J. G. Villegas, C. Prins, C. Prodhon, A. L. Medaglia, and N. Velasco. GRASP/VND and multi-start evolutionary local search for the single truck and trailer routing problem with satellite depots. *Engineering Applications of Artificial Intelligence*, 23(5):780–794, 2010. https://doi.org/10.1016/j.engappai.2010.01.013.

[2] J. G. Villegas. Original STTRPSD benchmark, 32 instances and format README. Author's download page: https://juangvillegas.com/download/. VRP-REP dataset 2022-0001: https://vrp-rep.org/datasets/item/villegas-et-al-2010.html.

OPTIONAL REFERENCE

Understanding the integer model

A OPTIONAL REFERENCE ON INTEGER PROGRAMMING

Use this appendix if you want to connect the routing rules to ideas from linear programming. The greedy constructor, local search, and metaheuristic can all use the explicit route representation from Section 2. You are not required to build or run an integer-programming solver.

A.1 Choices represented by binary variables

A linear programme often uses continuous variables to describe quantities, such as how much material to send along an edge. Here, some decisions are yes-or-no choices: use a truck trip, visit a satellite, or traverse a main-tour arc. A *binary variable* represents such a choice with value 1 for yes and 0 for no. If a variable selecting a trip were allowed to take value 0.5, it would select half a trip, which is not a valid routing decision.

Keep the distinction between data and decisions clear. Distances and customer demands are input data. The variables below record choices that an optimization method would make.

For each satellite $s \in \mathcal{S}$, let $\mathcal{R}_s$ contain all candidate truck trips that start and end at s, visit at least one customer, repeat no customer, and carry total demand at most Q_V. Let ℓ_r be the complete distance of candidate trip r. The constant a_{ir} is 1 when that trip contains customer i, and 0 otherwise. Define $W = \{0\} \cup \mathcal{S}$ and the possible directed main-tour arcs $A_1 = \{(u, v) \in W^2 : u \neq v\}$.

Variable	Meaning of value 1	Example from the toy solution
x_r	Select candidate truck trip r.	$x_{(1,3,4,1)} = 1$ serves customers 3 and 4 from satellite 1.
z_s	Visit satellite s on the main tour.	$z_1 = z_2 = 1$ visits both satellites.
y_{uv}	Traverse directed arc (u, v) on the main tour.	$y_{01} = y_{12} = y_{20} = 1$ encodes $(0, 1, 2, 0)$.

All these decisions are binary:

$$x_r, z_s, y_{uv} \in \{0, 1\}. \tag{1}$$

For the example trip $(1, 3, 4, 1)$, $a_{3r} = a_{4r} = 1$, whereas $a_{5r} = a_{6r} = 0$. These incidence values are known once the candidate trip is specified. Only x_r decides whether to use it.

A.2 Adding the distances of selected movements

The objective is

$$\min \sum_{(u,v)\in A_1} d_{uv} y_{uv} + \sum_{s\in\mathcal{S}} \sum_{r\in\mathcal{R}_s} \ell_r x_r. \tag{2}$$

The first sum adds a main-tour arc's distance when its variable is 1. The second adds a complete truck trip's distance when that trip is selected. A variable equal to 0 contributes nothing. For the toy solution, the first term is $d_{01} + d_{12} + d_{20} = 16$, and the three selected truck trips contribute the same distances calculated earlier. No extra cost is attached to selecting a satellite.

A.3 Serving each customer once and respecting capacity

For every customer i, impose

$$\sum_{s\in\mathcal{S}} \sum_{r\in\mathcal{R}_s} a_{ir} x_r = 1, \qquad i \in \mathcal{C}. \tag{3}$$

The left side counts selected trips containing customer i. Setting it to 1 simultaneously prevents omission and repeated service. Selecting two trips that both contain customer 3 would give a value of 2 for that customer's equation and would be forbidden.

Truck capacity is enforced when defining $\mathcal{R}_s$: every candidate in this set already satisfies its per-trip bound. In the toy example, $(1, 3, 4, 1)$ is eligible because $6 + 4 = 10 \leq Q_V$, whereas $(2, 5, 6, 2)$ is excluded because $8 + 7 = 15 > Q_V$. This explains why there is no additional capacity inequality in the displayed formulation. The input also satisfies $\sum_i q_i \leq Q_V + Q_T$. Several selected trips may share a satellite, and their combined demand need not fit in one truck load.

A.4 Linking truck trips to the main tour

A trip rooted at s can be selected only if the main tour visits s:

$$x_r \leq z_s, \qquad s \in \mathcal{S},\ r \in \mathcal{R}_s. \tag{4}$$

If $z_s = 0$, this inequality forces every corresponding x_r to 0. If $z_s = 1$, any number of feasible trips may be selected at that satellite, provided that the customer-coverage equations are satisfied. The one truck executes those trips sequentially. A visited satellite with no assigned trip is permitted and its main-tour travel is charged, as in the route-based definition.

Each selected satellite must have one outgoing and one incoming main-tour arc:

$$\sum_{v \in W \setminus \{s\}} y_{sv} = z_s \qquad s \in \mathcal{S}, \tag{5}$$

$$\sum_{u \in W \setminus \{s\}} y_{us} = z_s \qquad s \in \mathcal{S}. \tag{6}$$

When $z_s = 0$, both sums are zero and the main tour cannot visit s. The depot is always visited, so it has one departure and one return:

$$\sum_{v \in \mathcal{S}} y_{0v} = 1, \qquad \sum_{u \in \mathcal{S}} y_{u0} = 1. \tag{7}$$

A.5 Why the degree equations do not ensure one tour

Having one predecessor and one successor at each selected node does not ensure connection to the main depot. With three satellites a, b, c, the separate cycles $(0, a, 0)$ and (b, c, b) satisfy all those degree equations. The truck and trailer cannot execute the second cycle as part of a journey from depot 0 without adding other travel.

To rule out such disconnected cycles, take any nonempty subset B of satellites and impose

$$\sum_{u \in B} \sum_{v \in W \setminus B} y_{uv} \geq z_s, \qquad \varnothing \neq B \subseteq \mathcal{S},\ s \in B. \tag{8}$$

The left side counts main-tour arcs leaving B. If any satellite s in B is selected, the inequality requires at least one such arc. For the disconnected cycle above, choosing $B = \{b, c\}$ gives zero leaving arcs while $z_b = 1$, so the inequality fails. Requiring this for every subset prevents any selected cycle from remaining disconnected from depot 0. Together with the degree equations, it leaves a single depot-rooted main tour.

A.6 Using the formulation to understand a heuristic

The equations describe exactly the same feasible routes as the direct definition. A feasible route solution selects its main-tour arcs and truck trips, giving binary values that satisfy the constraints and the same distance. In the other direction, selected variables satisfying all the constraints give one connected main tour, capacity-feasible truck trips rooted on that tour, and exactly one service for each customer.

The family $\mathcal{R}_s$ can be extremely large because it contains every eligible ordered customer subset. There are also many possible satellite subsets in the connectivity conditions. Enumerating these families is not part of the project. Your metaheuristic constructs and changes only the candidate routes it encounters during search. The formulation can help you ask concrete questions about an operator: does it preserve unique customer coverage, respect capacity, keep every trip rooted at a visited satellite, and maintain one connected main tour? The scorer checks these properties directly on the submitted routes.